Forecasting Indoor Temperature One Hour Ahead from a Monash Environmental Sensor

MMA3001 Numerical Methods and Machine Learning, individual project. Edmund Leahy 33880867

Summary. I tested whether a simple linear regression could forecast the temperature one hour ahead better than assuming it stays the same. On the selected sensor over a 22-day test period, the regression reduced mean absolute error from 0.380°C to 0.365°C and root mean squared error from 0.697°C to 0.607°C. The MAE gain is small, at well under the 0.25 °C typical step between recorded values. All of it comes from the 7–9 am period, when a sharp temperature rise occurs, outside those hours' persistence is slightly better.

1. Engineering problem

Mechanical engineers design and operate the heating, ventilation and air-conditioning systems that determine how buildings behave thermally. Modern buildings carry networks of sensors, and a practical question is what those sensors can tell us about the near future without a physical model of the room.

The problem: How accurately can indoor temperature be predicted one hour ahead using only recent readings from the same sensor, and does a lagged linear regression outperform a persistence forecast?

Why: A short forecast is only worth building if it beats the free alternative of assuming nothing changes. Over short horizons temperature is strongly autocorrelated, so persistence is a demanding baseline and a useful test of whether a model adds anything.

Intended use: A monitoring aid for one sensor's location: an estimate of the temperature an hour from now, in °C.

Limits of the scope: this is a sensor-specific statistical forecast. It is not a control system, and it does not show energy savings, occupant comfort or transfer to other rooms, buildings or seasons.

2. Inputs, outputs and data

Table 1. Top-level inputs and outputs. Function-level detail is in the generated HTML documentation (docs/index.html).

Item	Specification
Raw input	supplied by the unit as part of dataset 2. One CSV with a JSON array in the jsondata column.
Model inputs	T(t), T(t−20 min) and T(t−40 min), in °C, from one sensor. Observed domain on the selected sensor.
Output	Forecast temperature $\hat{T}$(t+60 min) in °C, plus saved metrics and figures.
Time	Event time from the nested values. Melbourne local time is used only for hour-of-day analysis and plot axes.
Invalid input	Bad JSON, missing or non-numeric values and implausible timestamps are counted by reason and skipped. Examples with any missing input or target are excluded and counted; gaps are never interpolated. Asking for an unknown sensor or a horizon that is not a multiple of the time step raises an error.

Data audit

I audited the full file before choosing anything. There were no parse errors and no conflicting duplicates, but the file differed from what its name suggests.

- **Row count:** The 179,447 rows hold only 42,843 unique temperature readings, because most rows repeat an earlier reading.
- **Unusable sensors:** Three of the five sensors have only 1 or 2 unique readings each, despite having thousands of rows.
- **Stopped sensor:** Sensor 6012002000869 (“869”) stopped producing new readings on 14 June. Its last reading was still re-sent 15431 times, with ingestion dates up to October 2024, so I used event time rather than ingestion time.
- **Dates:** Event times run from May 2023 to April 2026. Only the May to June 2024 part of 869 falls within “MayToDec2024”.

Table 2. The two usable sensors. Locations are from the supplied workbook, Sensor ID and Locations.xlsx.

Sensor	Listed location	Period (local)	Unique readings	20-min slots filled
869	Building 60, G.38	29 Feb to 14 Jun 2024	13,800	99%
326 (Secondary)	N/A	11 Sep 2025 to 22 Apr 2026	29,037	97%

Selected sensor: I chose 869 because it is the only one of the five with a verified location, and the question is about indoor temperature. Plus its longest gap is 30 minutes.

Second sensor: Sensor 326 has twice the data but a 4.5-day outage, a much wider range (11.8–37.5 °C) and an unknown location, so I only used it as an independent check.

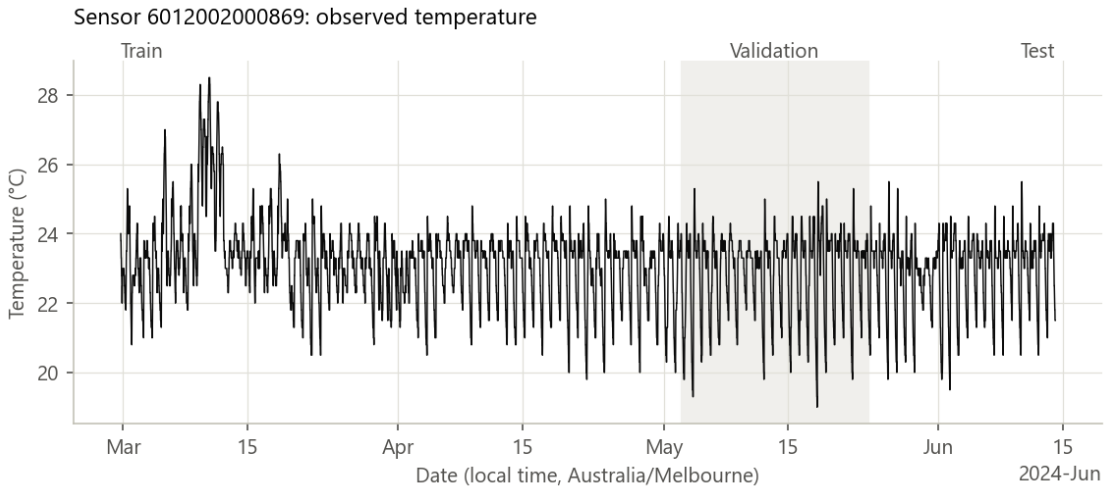


Figure 1. Sensor 869 on the 20-minute grid.

3. Computational approach

Time grid: Readings arrive about every 10 minutes, but about 1 in 10 is missed. So I resampled them onto a 20 minute grid, where each slot holds the latest reading at or before the slot’s end time. That value was already known at that time, so no future information enters the inputs. Empty slots stay empty.

Baseline: Persistence assumes the temperature in one hour equals the latest reading, $\hat{T}(t+H) = T(t)$. It has no parameters.

Selected method: Linear regression, fitted by ordinary least squares on the training period. This finds the coefficients that minimise the sum of squared errors. The fitted model is:

$$\hat{T}(t+60\ \text{min}) = 2.807 + 1.650 \cdot T(t) - 0.520 \cdot T(t-20\ \text{min}) - 0.251 \cdot T(t-40\ \text{min})$$

What the model does: The large weight on the current value and the negative weights on older values continue the recent trend. The coefficients sum to 0.88 rather than 1, which pulls forecasts back towards about 23 °C, close to the sensor's 23.3 °C median.

Why: Regression is taught in the unit, is cheap and has coefficients that can be read and checked.

Alternatives: Decision trees, support vector regression and neural networks were considered but not implemented. With so little gain available from past temperature alone, new inputs are probably a more useful next step than a more flexible model.

Assumptions: The next value depends linearly on the last 40 minutes of readings. The relationship learned in March-May still holds in late May-June. The sensor is accurate enough for changes of about 0.25 °C to mean something.

Scaling: None is used, because all inputs are in °C and scaling does not change a least-squares prediction.

Numerical behaviour: scikit-learn solves the problem directly with an SVD-based routine, so there is no iteration or convergence criterion. Neighbouring temperatures are strongly correlated, so I measured the condition number of the standardised inputs: 20 for the selected model. This is not a numerical problem in double precision, although individual coefficients are less stable than the predictions.

Implementation: Python, using NumPy, scikit-learn and matplotlib, with one command for each stage (audit, select, final, profile). Exact commands and package versions are in the README.

4. Validation method and results

Verification: does the code do what it specifies?

All 63 pytest tests on synthetic data pass (results/test_report.txt). They check:

- **Data handling:** Parsing, bad-record counting, UTC-to-Melbourne conversion across daylight saving, and deduplication.
- **Timing:** The target falls exactly one horizon ahead, even across gaps. Features do not change when future values change. No grid slot is shared between time periods.
- **Calculations:** Metrics match hand calculations. Regression coefficients match a known model and an independent NumPy solution.
- **Separation:** Rewriting the test period by +50 °C does not change model selection.

Audit check: A separate check during development, using only Python's standard library and not kept in the repository, reproduced the audit's per-sensor counts from the raw ZIP.

Validation: do predictions hold on unseen real data?

Split: Chronological 60/20/20, and an example is kept only if its whole window, from oldest input to target, lies inside one period. Of 7,569 complete examples, 28 were removed at the boundaries. A random split would be inappropriate because neighbouring readings are nearly identical, which would make error look better than it really is.

Metrics: Mean absolute error (MAE) is the average size of the error in °C. Root mean squared error (RMSE) weights large errors more heavily.

Uncertainty: I report the MAE gain with a 95% interval, from a bootstrap of 2,000 resamples of whole local days. Whole days are used because errors close together in time are correlated, and resampling single rows would overstate the evidence.

Model selection: Validation data chose the number of inputs and informed the sensitivity comparisons. It was scored once after the settings were locked, and the code refuses to overwrite that result without an explicit reproduce flag.

Table 3. Results for sensor 869, using a model fitted on the training period only.

Partition	Examples	Persistence MAE (°C)	Regression MAE (°C)	Persistence RMSE (°C)	Regression RMSE (°C)
Train, 29 Feb – 2 May 2024	4,544	0.328	0.321	0.532	0.475
Validation, 3–24 May 2024	1,492	0.375	0.349	0.625	0.526
Test, 24 May – 14 Jun 2024	**1,505**	**0.380**	**0.365**	**0.697**	**0.607**

Result: The regression improved test MAE by 4.0% and RMSE by 12.9%. The MAE gain was 0.015 °C (95% interval 0.002 to 0.030 °C, over 22 days).

Meaning: The interval excludes zero, so the gain is probably real. It is only about one-seventeenth of the 0.25 °C median step between recorded values, so on its own it means little in practice.

Overfitting: Training error is not far below test error, so there is no sign of serious overfitting.

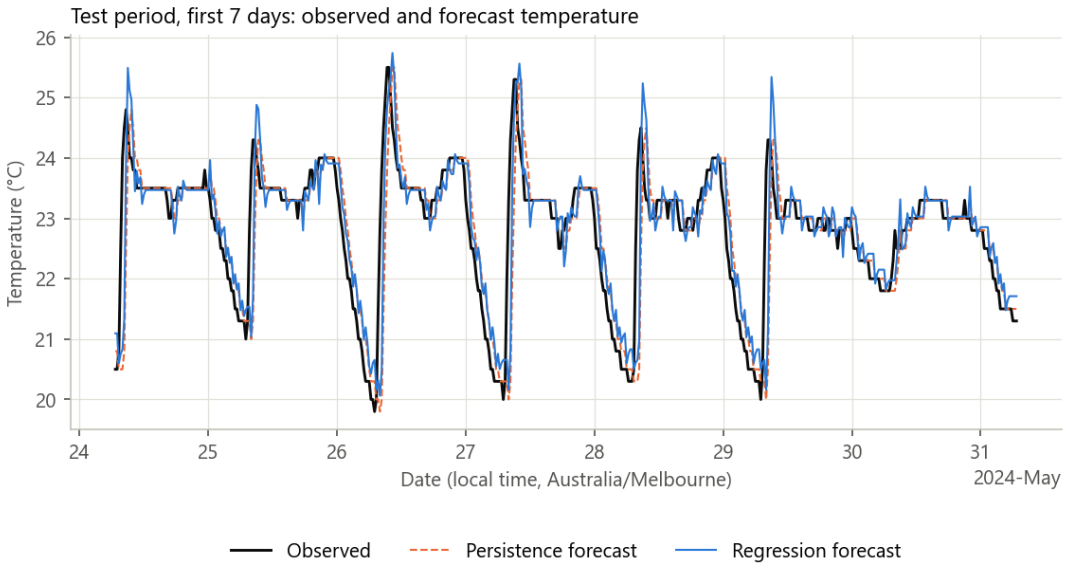


Figure 2. First seven days of the test period.

Where the error occurs: The 7–9 am hours hold 40% of persistence's total error and 37% of the regression's (Figure 3). At 8 am, MAE is 2.42 °C for persistence and 1.87 °C for regression.

Other hours: Averaged over the other 21 hours, persistence is slightly better (0.258 °C against 0.264 °C), and it wins every hour from 1 pm to midnight. When temperature is stable, persistence is often exactly right, while the regression is rarely exact (Figure 4).

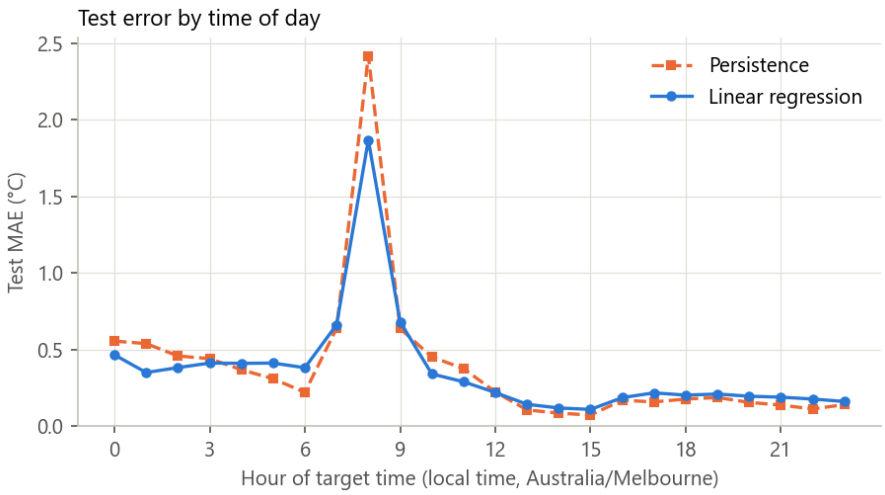


Figure 3. Test MAE by local hour of the forecast's target time, sensor 869.

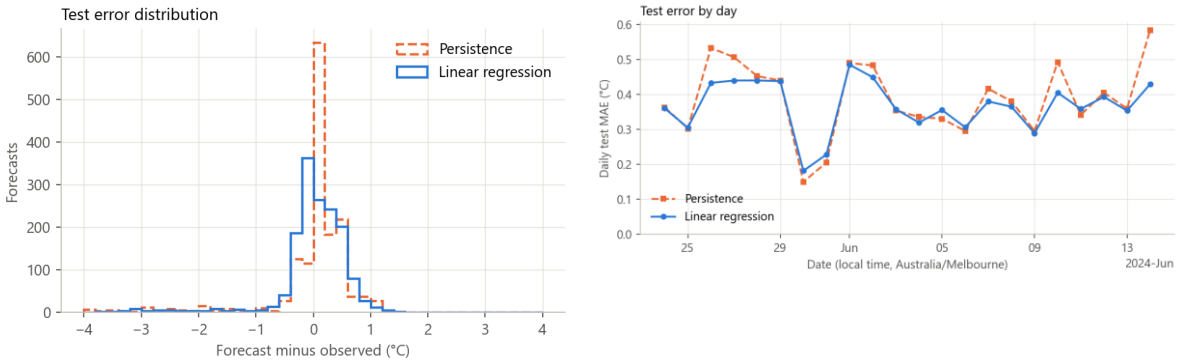


Figure 4. Left: test error distribution; persistence has a tall peak at zero and more large errors.

Second sensor: The same settings on sensor 326, with its own training and test periods, gave test MAE of 0.444°C for persistence and 0.348°C for regression. RMSE was 0.605°C against 0.508°C. The method carries over to a second, more variable signal, but the sensor's location is unknown, so this is not a second indoor result.

What this cannot show: Performance in other seasons, rooms or operating conditions, or whether errors of this size are acceptable for a specific application. The 22-day test period is also short, and the bootstrap treats days as independent, which is only an approximation.

5. Comparison and optimisation

All settings below were compared on the validation period of sensor 869.

Table 4. Validation sensitivity. Persistence MAE on the rows used for the lag comparison is 0.375°C. The condition number is for the standardised inputs.

Number of inputs	Inputs' span	Regression MAE (°C)	Regression RMSE (°C)	Condition number
1	Current only	0.398	0.605	1
3 (selected)	**40 min**	**0.349**	**0.526**	**20**
6	100 min	0.349	0.525	31
12	220 min	0.348	0.522	43

Number of inputs: My first rule, lowest validation MAE, chose 12 inputs. The gain over 3 inputs was only 0.001 °C, far below what the sensor can resolve, and the condition number doubled.

Rule change: I changed the rule to "the fewest inputs within 0.01 °C of the best", which selects 3. I did this after seeing validation results but before any test scoring, so the test is not affected.

One input: A single input is worse than persistence on MAE (0.398°C against 0.375°C) but better on RMSE (0.605°C against 0.625°C). This fits a model that pulls forecasts towards the mean, which reduces large errors but loses the frequent exact hits.

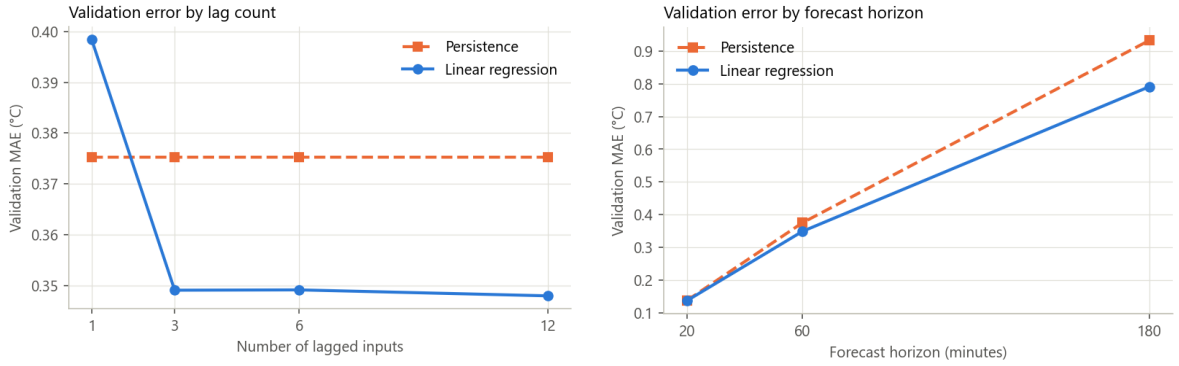


Figure 5. Left: validation MAE by number of inputs. Right: validation MAE by forecast horizon, with 3 inputs.

Table 5. Other validation comparisons with 3 inputs or an equivalent history of about 1 hour.

Setting	Training examples	Persistence MAE (°C)	Regression MAE (°C)	MAE improvement
Horizon 20 min	n/a	0.138	0.137	0.4%
Horizon 60 min, step 20 min	**4,553**	**0.375**	**0.349**	**7.0%**
Horizon 180 min	n/a	0.932	0.791	15.2%
Step 10 min, 6 inputs	3,585	0.397	0.381	4.0%
Step 30 min, 2 inputs	3,035	0.377	0.346	8.2%

Horizon: This had the largest effect. Error grows about sixfold from 20 to 180 minutes, and the regression's advantage grows with it. At 20 minutes it adds almost nothing.

Time step: 10 minutes leaves fewer complete examples because missed readings break more input windows. A 30-minute step scored slightly better than 20 minutes (0.346 °C against 0.349 °C), but on about a third fewer examples and different rows, so I kept 20 minutes.

Table 6. Computational cost on 4,544 training rows, with 3 inputs. Times are medians of 20 runs on one laptop. FLOPS are estimates, not measurements.

Measure	Persistence	Linear regression
Fit time	None	1.4 ms
Predict all rows	0.02 ms	0.6 ms
Peak Python allocation during fit	None	0.41 MB
Estimated FLOPS to fit ($2np^2 - \frac{2}{3}p^3$)	0	about 82,000
Estimated FLOPS per prediction ($2p$)	0	6

Performance: Cost does not limit either method. Profiling with cProfile shows half of the 0.96s selection run goes on saving figures, so optimising the model code would achieve little. The slowest stage is the audit, at about 12–13 s to stream the full archive without extracting the 353 MB CSV.

Final design: Persistence is cheaper, simpler and better during stable hours. The regression is better overall and clearly better during the morning rise. I selected the 3-input regression because it reduces persistence's large errors at negligible cost and is simple enough to inspect.

6. Limitations and lessons learned

- **Scope:** One primary sensor, about 3.5 months of one season, with a 22-day test period. The space above the compactus could have little activity during this period or an isolated airspace so a busier room could have more variance.
- **Morning rise:** The cause is unknown. It recurs on most days shown in Figure 2, which may reflect building operation, like people arriving.
- **Inputs:** Only past temperature is used, so the model cannot anticipate a rise that has not started. Time of day, humidity and CO2 were not tested. Time of day is the obvious next experiment, since the largest error occurs at a predictable hour.
- **Resolution and timing:** Values are recorded in steps of about 0.25 °C. Because a slot holds the latest reading in that slot, the true gap between input and target can differ slightly from a nominal 60 minutes.
- **Fixed model:** Coefficients are not updated as conditions change.

Lessons

- **Audit first:** The filename and row count both suggested far more data than was there.
- **Baselines:** Persistence set a bar that one-input regression failed to clear.
- **Breakdowns:** The overall score hid the main behaviour. The gain comes from about three hours a day, which only the hourly breakdown showed.
- **Metrics:** MAE and RMSE can disagree, and understanding why gave more insight than either number alone.
- **Honest results:** A small MAE gain is still a valid result, and reporting it plainly is more useful than presenting 4% as a success.

7. AI use and reflection

Tools and level of contribution

- **Claude:** Claude Code (model Opus 5.5) wrote some code and README, as well as formatting the report to flow. Laid out the project process and order of tasks to be done, was able to progress the project.
- **ChatGPT:** Used to collate information and create prompts for Claude. Was able to also give formatting suggestions and information on the report as well as correct code errors.

Decisions

Claude presented evidence and recommendations. I decided on sensor 869, the 20 minute step, the 60 minute horizon, using sensor 326 as a secondary test, number of inputs, type of outputs, what to provide in the report.

Errors and limitations found in the AI's work

- **10-minute grid:** Claude first did not accept the 10 minute grid as there was missing data and it created more errors. The model worked better at a 20 minute step.
- **Crash:** The first validation run crashed on the 10 minute setting because the missing period was empty. The code now reports zero examples, and a test was added.
- **Overcorrection and overreaching:** AI would often do more than was required and would change too much at once outside of what I asked and it made it more difficult.

- **No common sense:** No layer of common sense in the models when it comes to assessing data and what it means. So it would often reject data or not understand trends that are obvious to me.

Reflection

- The AI is very powerful tool in coding, able to complete large amounts in very short time.
- I didn't use the outputs from the AI ever only running it on my own. I'm sure I could have used the AI more and compared but I just ran it myself.
- I have a much better understanding on workflow and how to work with AI. I am able to collaborate much better and get better work from the AI.

References

Efron, B. and Tibshirani, R.J. (1993) An Introduction to the Bootstrap. Chapman & Hall.

Golub, G.H. and Van Loan, C.F. (2013) Matrix Computations, 4th edn. Johns Hopkins University Press.